Facultad de Ingenierías

PROGRAMACIÓN DE SISTEMAS WEB

Actividad Integradora 3

Inventario de Productos: módulo web CRUD con PHP, PDO y MySQL con sentencias preparadas

Informe técnico

Estudiante: Angel Mauricio Guevara Chinchero

Carrera: Ingeniería en Sistemas Inteligentes

Docente: Gregorio Cruz Gonzabay

Repositorio: https://github.com/aguevaranv/AI3_DisenoWeb_AngelGuevara

Fecha de entrega: 08 de noviembre de 2026

1. Introducción

Inventario de Productos es un módulo web full-stack desarrollado en **PHP 8** con una base de datos **MySQL/MariaDB** (XAMPP). Permite **registrar** productos mediante un formulario validado (Create) y **consultarlos** en una tabla responsiva (Read). El acceso a datos se hace con **PDO** y sentencias preparadas, la salida se sanea con `htmlspecialchars()` y el código está organizado en módulos: configuración, conexión, modelo, funciones de ayuda y vistas.

2. Sección 1: Justificación técnica

2.1 Análisis comparativo: PDO vs MySQLi

PHP ofrece dos extensiones para trabajar con MySQL. Ambas admiten sentencias preparadas y programación orientada a objetos, pero difieren en arquitectura, alcance y forma de manejar los errores:

Aspecto	PDO (PHP Data Objects)	MySQLi (MySQL improved)
Arquitectura	Capa de abstracción con controladores para varios motores (MySQL, PostgreSQL, SQLite, SQL Server, Oracle…).	Extensión específica de MySQL/MariaDB.
Estilo	Solo orientado a objetos.	Orientado a objetos y procedural (mysqli_query()).
Marcadores	Con nombre (`:codigo`) y posicionales (`?`).	Solo posicionales (`?`).
Enlace de datos	`bindValue()` / `bindParam()` o un arreglo en `execute()`.	`bind_param("ssdi", ...)`, con una letra por tipo.
Errores	`PDOException` con `ERRMODE_EXCEPTION` (modo por defecto desde PHP 8).	`mysqli_sql_exception` (por defecto desde PHP 8.1).
Rendimiento	Prácticamente igual en un CRUD; la diferencia la marca la base de datos, no la extensión.	Ligeramente más rápido en pruebas masivas; acceso a funciones propias de MySQL.
Seguridad	Sentencias preparadas nativas si se desactiva la emulación.	Sentencias preparadas nativas.

2.2 Extensión seleccionada: PDO

- **Portabilidad:** si el proyecto migra a PostgreSQL o SQLite, solo cambia la cadena DSN de `conexion.php`; el modelo y las consultas estándar se mantienen.
- **Abstracción:** una sola API orientada a objetos y marcadores con nombre, que hacen el SQL más legible (`:nombre` en lugar de contar signos `?`).
- **Manejo de excepciones:** con `PDO::ERRMODE_EXCEPTION` todos los errores de conexión y de SQL se capturan con `try-catch`, lo que permite registrar el detalle y mostrar al usuario un mensaje genérico.

2.3 Medidas de seguridad implementadas

Prevención de inyección SQL. Toda consulta que recibe datos del usuario usa `prepare()` con marcadores y `bindValue()` / `execute()`. Además, la conexión desactiva la emulación (`PDO::ATTR_EMULATE_PREPARES => false`), por lo que MySQL recibe primero la consulta y después los valores por separado: un texto como `x'); DROP TABLE productos; --` se guarda como dato y nunca se ejecuta.

Prevención de XSS. Todo valor impreso en HTML pasa por la función `e()`, que convierte `< > & " '` en entidades HTML. Así, un nombre como `<script>alert(1)</script>` se muestra como texto:

```
function e(?string $valor): string
{
    return htmlspecialchars($valor ?? '', ENT_QUOTES, 'UTF-8');
}
```

Protección de credenciales y mensajes del sistema. Las credenciales están en `config/database.php`, que está en el `.gitignore`; en GitHub solo se publica la plantilla `database.example.php`. Si la conexión falla, el detalle se guarda con `error_log()` y se lanza un mensaje genérico:

```
try {
    return new PDO($dsn, $config['user'], $config['password'], [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ]);
} catch (PDOException $e) {
    // El mensaje real puede mostrar el usuario o el servidor: solo va al log.
    error_log('Error de conexión: ' . $e->getMessage());
    throw new RuntimeException('No se pudo conectar con la base de datos.');
}
```

Validación en el cliente y en el servidor. El formulario usa atributos HTML5 (`required`, `pattern`, `min`, `maxlength`) y `validacion.js` para avisar antes de enviar. Como esa validación se puede saltar, `validarProducto()` vuelve a revisar todo en PHP (formato `ABC-123`, longitud con `mb_strlen()`, categoría con `in_array()`, enteros con `FILTER_VALIDATE_INT`) y responde con el código HTTP 422 si hay errores.

Patrón POST-Redirect-GET. Tras guardar, `crear.php` redirige con el código 303 a `index.php`; recargar la página no reenvía el formulario ni duplica registros.

3. Sección 2: Arquitectura del sistema

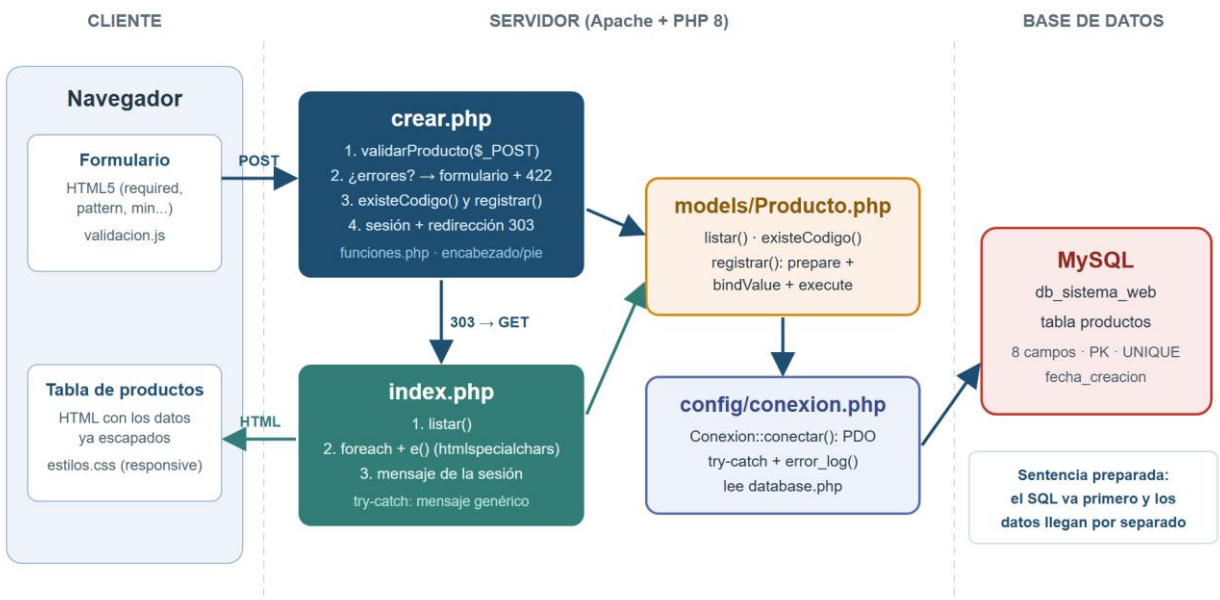


Figura 1. Flujo de datos entre el cliente, los scripts PHP y la base de datos MySQL.

3.1 Flujo de datos

- **Registro (Create):** el navegador valida el formulario y lo envía por POST a `crear.php`. El script valida los datos, verifica con `existeCodigo()` que el código no exista y llama a `registrar()` del modelo, que ejecuta el INSERT preparado a través de la conexión PDO. Luego guarda un mensaje en la sesión y redirige con 303.
- **Consulta (Read):** el navegador pide `index.php` por GET. El script obtiene la conexión, llama a `listar()` y recorre los resultados con `foreach`, escapando cada valor con `e()` antes de devolver el HTML de la tabla.

3.2 Responsabilidad de cada módulo

Archivo	Responsabilidad
`config/database.php`	Credenciales locales (arreglo devuelto con return). Ignorado por Git.
`config/conexion.php`	Clase Conexion: crea el objeto PDO con sus opciones y maneja el error de conexión.
`models/Producto.php`	Único archivo con SQL: listar(), existeCodigo() y registrar().
`includes/funciones.php`	e() para escapar la salida y validarProducto() para la validación del servidor.
`includes/encabezado.php`, `pie.php`	Estructura HTML y menú compartidos por las dos páginas.

Archivo	Responsabilidad
`crear.php / index.php`	Controlan cada página: procesan la petición y muestran la vista.
`js/validacion.js, css/estilos.css`	Validación en el cliente y diseño responsivo.
`database/db_sistema_web.sql`	Script DDL para crear la base y la tabla.

3.3 Diseño de la base de datos

La tabla `productos` de `db_sistema_web` tiene 8 campos. Cumple la tercera forma normal: cada celda guarda un solo valor y todos los campos dependen solo de la clave primaria.

Campo	Tipo	Justificación
`id`	INT UNSIGNED AUTO_INCREMENT	Clave primaria generada por MySQL.
`codigo`	VARCHAR(7) UNIQUE	Formato ABC-123; el índice único impide duplicados.
`nombre`	VARCHAR(80)	Texto corto de largo variable.
`categoria`	VARCHAR(30)	Valor de una lista cerrada validada en PHP.
`precio`	DECIMAL(10,2)	Valor monetario exacto (FLOAT acumula errores de redondeo).
`stock`	INT UNSIGNED DEFAULT 0	Cantidad entera y no negativa.
`descripcion`	VARCHAR(255) NULL	Campo opcional.
`fecha_creacion`	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Auditoría: fecha y hora de registro automáticas.

4. Sección 3: Evidencias de ejecución

4.1 Estructura de la tabla en phpMyAdmin

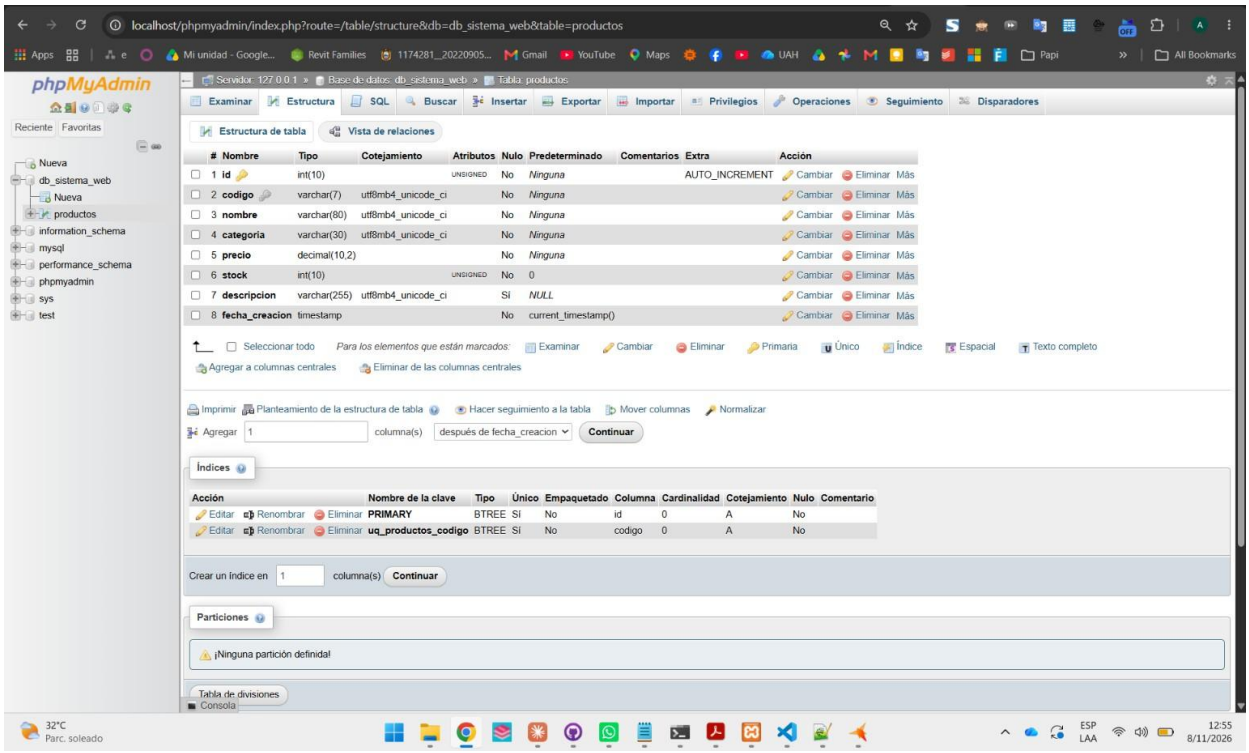


Figura 2. Estructura de la tabla productos en phpMyAdmin.

La captura muestra los 8 campos con sus tipos: `id` con `AUTO_INCREMENT` y atributo `UNSIGNED`; `precio` como `decimal(10,2)`; `descripcion` como único campo que admite NULL, y `fecha_creacion` con valor predeterminado `current_timestamp()`. En la sección Índices aparecen la clave primaria (`PRIMARY` sobre id) y el índice único `uq_productos_codigo`, que garantiza que no se repitan códigos.

4.2 Conexión a la base de datos

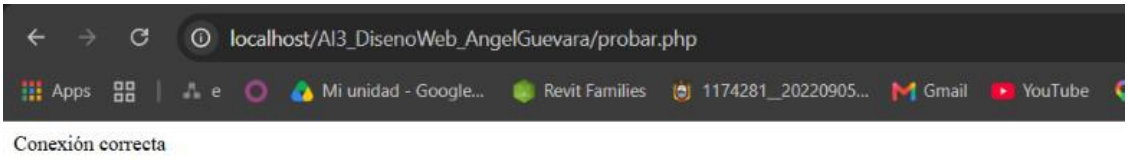


Figura 3. probar.php confirma la conexión PDO con db_sistema_web.

El script `probar.php` llama a `Conexion::conectar()` dentro de un `try-catch`. El mensaje "Conexión correcta" confirma que Apache, PHP y MySQL están integrados y que las credenciales de `config/database.php` son válidas. Si fallara, solo se mostraría el mensaje genérico, nunca el error de PDO.

4.3 Código con sentencias preparadas

Método `registrar()` de `models/Producto.php`:

```
public function registrar(array $datos): bool
{
    $sql = 'INSERT INTO productos (codigo, nombre, categoria, precio, stock, descripcion)
            VALUES (:codigo, :nombre, :categoria, :precio, :stock, :descripcion)';

    $consulta = $this->db->prepare($sql);
    $consulta->bindValue(':codigo', $datos['codigo']);
    $consulta->bindValue(':nombre', $datos['nombre']);
    $consulta->bindValue(':categoria', $datos['categoria']);
    $consulta->bindValue(':precio', $datos['precio']);
    $consulta->bindValue(':stock', (int) $datos['stock'], PDO::PARAM_INT);

    if ($datos['descripcion'] === '') {
        $consulta->bindValue(':descripcion', null, PDO::PARAM_NULL);
    } else {
        $consulta->bindValue(':descripcion', $datos['descripcion']);
    }

    return $consulta->execute();
}
```

Figura 4. Inserción con prepare(), bindValue() y execute() en models/Producto.php.

El SQL contiene solo marcadores (`:codigo`, `:nombre`…), nunca los datos del usuario. `prepare()` envía la consulta a MySQL; `bindValue()` asocia cada valor indicando su tipo (`PDO::PARAM_INT` para el stock y `PDO::PARAM_NULL` cuando la descripción está vacía), y `execute()` ejecuta la sentencia. Como los valores viajan separados del SQL, no pueden alterar la consulta.

4.4 Formulario en ejecución con datos de prueba

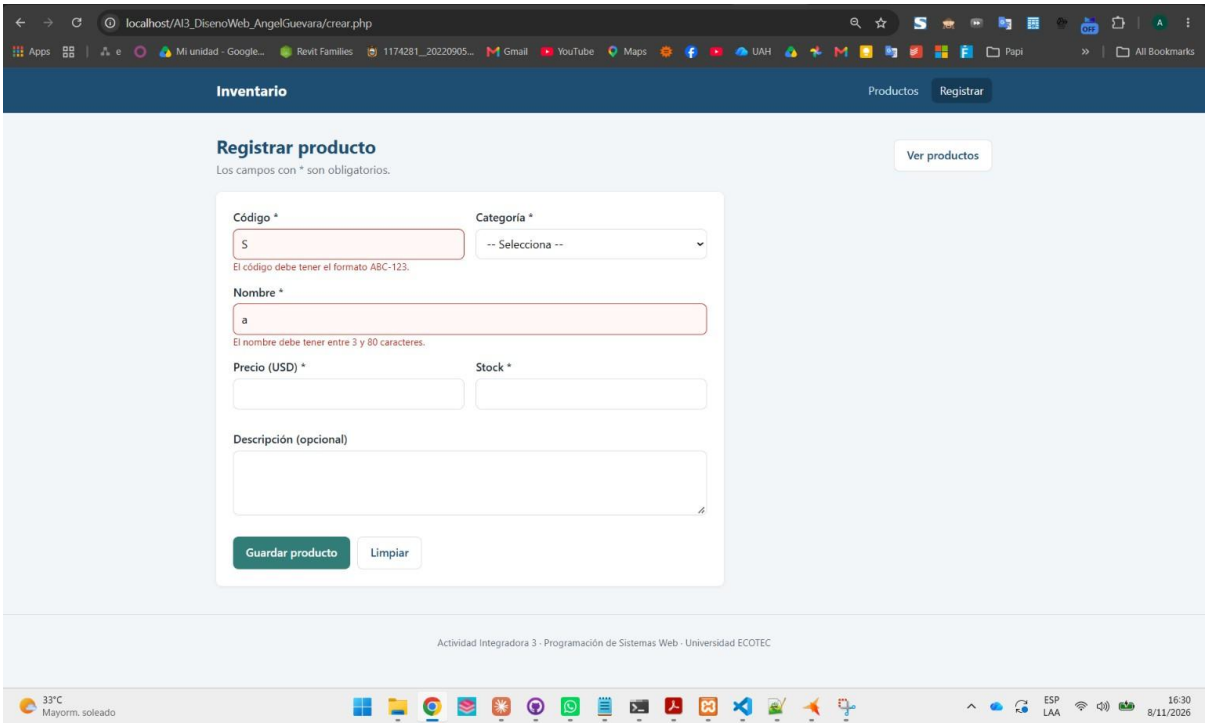


Figura 5. Formulario de registro (crear.php) validando datos de prueba incorrectos.

La captura muestra la validación en el cliente con datos de prueba incorrectos: el código "S" no cumple el formato `ABC-123` y el nombre "a" tiene menos de 3 caracteres. `validacion.js` revisa cada campo con `checkValidity()` mientras el usuario escribe, marca el campo en rojo y muestra el mensaje debajo. Los campos que todavía no se han tocado (precio y stock) aún no muestran error. Si se intenta guardar, el envío se cancela con `preventDefault()`, y si alguien saltara esta validación, `validarProducto()` la repetiría en el servidor.

4.5 Tabla con los registros insertados

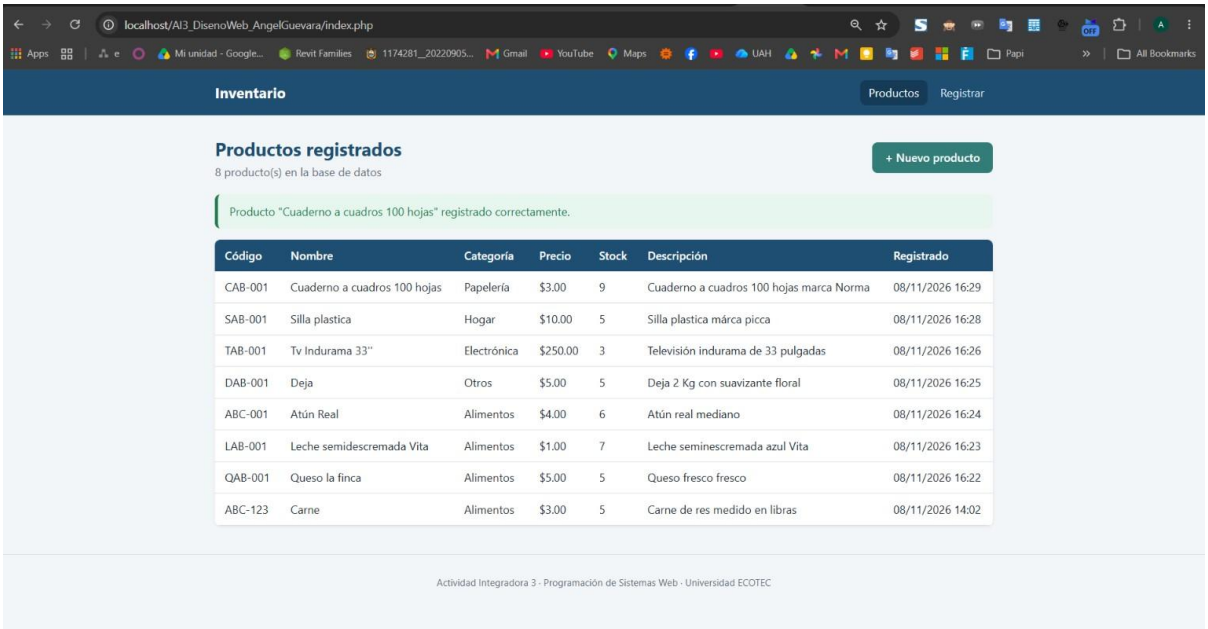


Figura 6. Tabla de productos (index.php) con los registros guardados en MySQL.

Después de registrar el producto "Cuaderno a cuadros 100 hojas", `crear.php` redirigió con el código 303 a `index.php`, que muestra el mensaje verde de confirmación guardado en la sesión. La tabla lista los **8 productos** obtenidos con `listar()`, ordenados del más reciente al más antiguo según `fecha_creacion`, la fecha que asignó MySQL automáticamente. Los precios aparecen con dos decimales gracias a `DECIMAL(10,2)` y `number_format()`, y cada texto se imprime escapado con `e()`; por ejemplo, las comillas de "Tv Indurama 33'"" se muestran como texto sin romper el HTML.

4.6 Diseño responsivo



Figura 7. Listado en celular: cada fila es una tarjeta.

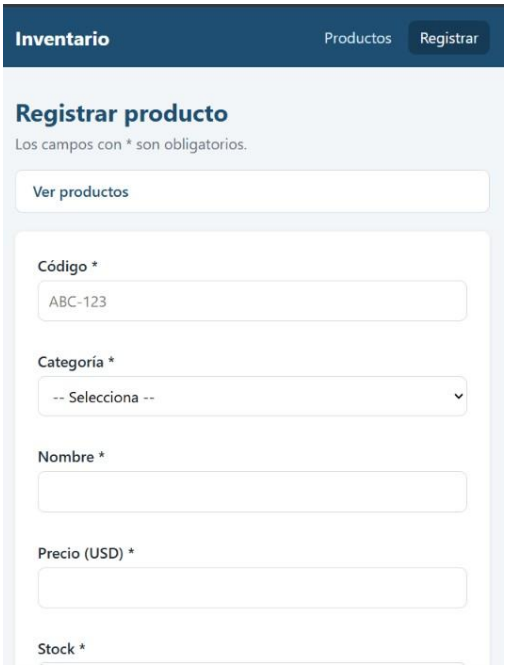


Figura 8. Formulario en celular: los campos en una columna.

En pantallas de menos de 700 px se aplica la media query de `estilos.css`: en el listado se oculta el encabezado de la tabla y cada fila se convierte en una tarjeta, con el nombre de cada dato tomado del atributo `data-label` mediante `td::before`. En el formulario, las filas de dos campos pasan a una sola columna con `flex-direction: column`, de modo que la página se puede usar en el celular sin desplazamiento horizontal.

4.7 Historial de commits

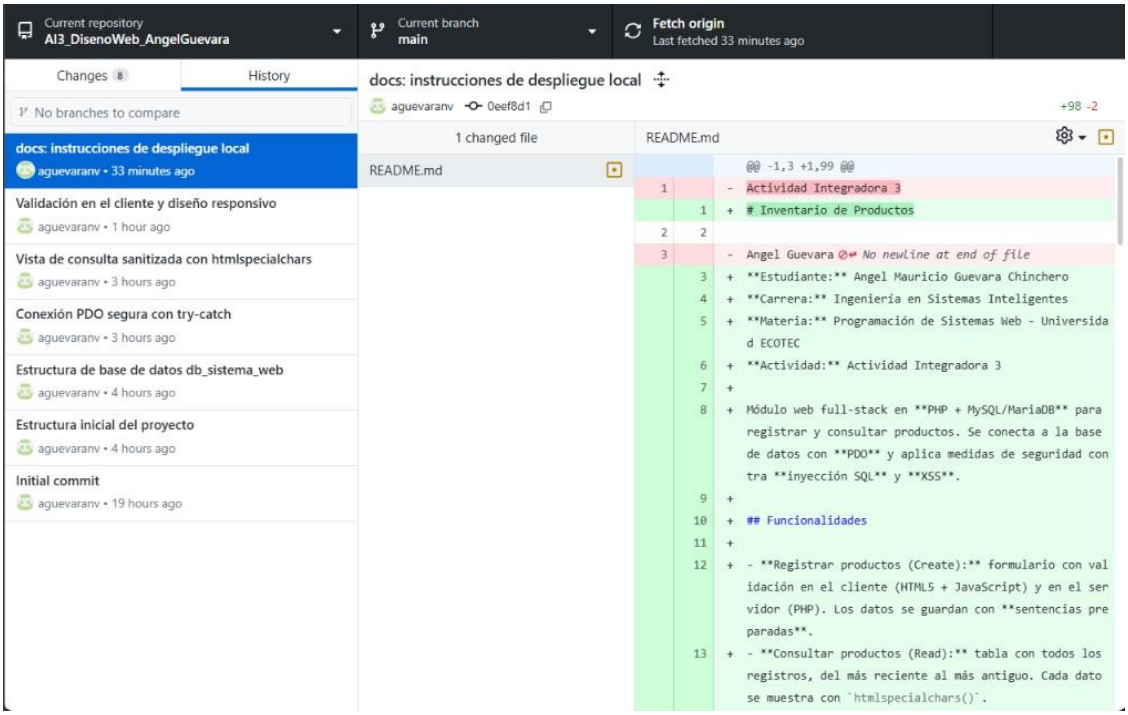


Figura 9. Historial de commits del repositorio en GitHub Desktop.

La pestaña History muestra los **7 commits** del proyecto, del más antiguo (Initial commit) al más reciente, cada uno con un avance concreto: estructura inicial con el `.gitignore`, script de la base de datos, conexión PDO segura con try-catch, vista de consulta sanitizada con htmlspecialchars, validación en el cliente con diseño responsivo y, por último, la documentación del despliegue. A la derecha se ve el cambio del último commit: el `README.md` pasó de 3 líneas a las instrucciones completas.

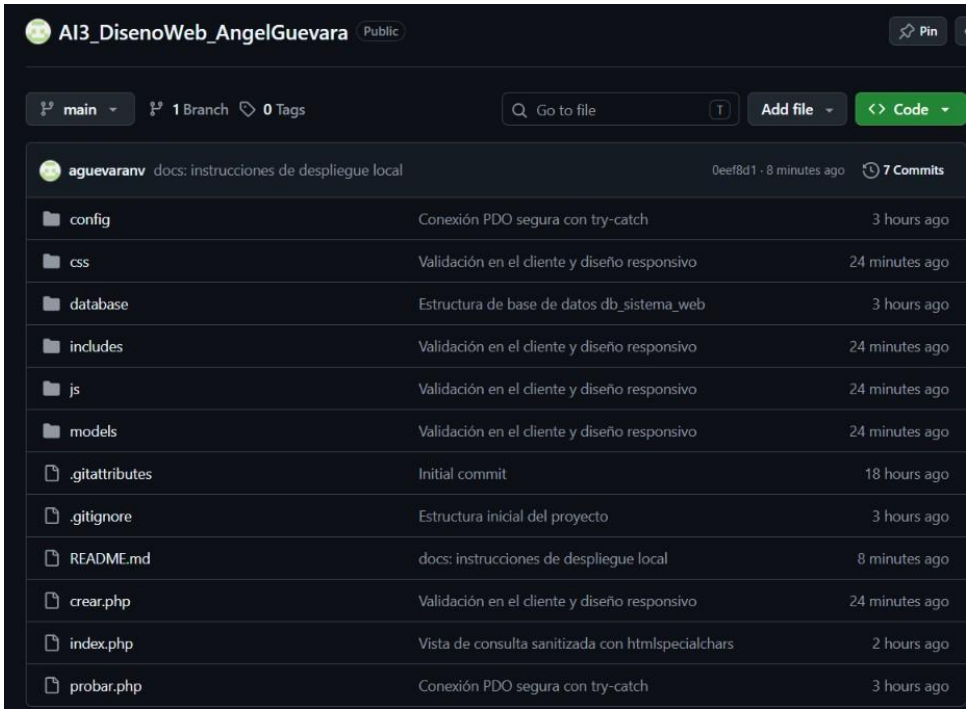


Figura 10. Repositorio público AI3_DisenoWeb_AngelGuevara en GitHub.

En GitHub el repositorio aparece como **público** con los mismos 7 commits, y cada carpeta muestra el último commit que la modificó. El archivo `config/database.php` no aparece porque está excluido por el `.gitignore`, así que las credenciales no se publican.

5. Sección 4: Enlaces del proyecto

Repositorio público en GitHub: https://github.com/aguevaranv/AI3_DisenoWeb_AngelGuevara

Las instrucciones para desplegar el proyecto en XAMPP están en el `README.md` de la raíz del repositorio.

6. Conclusión

El módulo integra PHP y MySQL de forma segura y ordenada. Elegir PDO permitió usar marcadores con nombre y un manejo uniforme de excepciones, y combinar sentencias preparadas con `htmlspecialchars()` cubre los dos ataques más comunes en aplicaciones web: la inyección SQL y el XSS. Separar la configuración, la conexión, el modelo y las vistas hizo que cada archivo tenga una sola responsabilidad, y el uso de Git con un .gitignore adecuado mantuvo las credenciales fuera del repositorio público.